

 BISA DARI HP

Upload ke GitHub Tanpa Komputer

Cara upload manual file TeleDrive ke GitHub langsung dari browser HP — tanpa install git, tanpa terminal.

0 Persiapan — Yang Kamu Butuhkan



Kamu butuh 2 hal saja:

1. Akun GitHub (daftar gratis di github.com)
2. File-file TeleDrive yang sudah disiapkan (bisa download dari preview panel)

Pastikan kamu sudah punya semua file di bawah ini. File-file ini ada di folder `deploy-vercel/`:



vercel.json

Konfigurasi Vercel

1 KB



package.json

Info project

0.3 KB



.gitignore

Git ignore rules

0.1 KB



api/tg-api.js

Proxy JSON ke Telegram

3 KB



api/tg-upload.js

Proxy upload file ke Telegram

6 KB



public/teledrive.html

436 KB



public/logo.svg

Logo

2 KB



public/robots.txt

SEO

0.1 KB



Cara download file-file ini:

Di preview panel, klik "Open in New Tab" lalu download satu per satu. Atau download `deploy-vercel.zip` lalu extract di HP kamu.

1

Cara Paling Mudah: Drag & Drop Folder

GitHub mendukung upload folder langsung. Ini cara tercepat — **1 kali upload untuk semua file.**

LANGKAH 1.1

Buat Repository Baru di GitHub

Buka browser di HP/laptop, pergi ke:

github.com/new

Isi form berikut:

Form di github.com/new

```
Repository name: teledrive
Description: TeleDrive - Telegram Cloud Drive
Public ☒ (pilih Public)
Add a README ☒ (JANGAN dicentang!)
Add .gitignore ☒ (JANGAN dicentang!)
License ☒ (Kosongkan / None)
```

Klik tombol hijau **"Create repository"**



Halaman "Create a new repository" di GitHub

Isi nama "teledrive", jangan centang apapun, klik Create



LANGKAH 1.2

Buka Halaman Upload

Setelah repository terbuat, kamu akan melihat halaman kosong. Klik:

Add file



Upload files



Di halaman repository kosong, klik "Add file" → "Upload files"



LANGKAH 1.3

Upload Folder deploy-vercel



GitHub bisa upload folder sekaligus!

Jika kamu sudah extract file dari ZIP di HP/laptop, cukup drag & drop **seluruh folder** `deploy-vercel` ke area upload di GitHub.



TAPI — ada masalah: GitHub akan membuat struktur:

Struktur yang TIDAK diinginkan

```
teledrive/  
  deploy-vercel/      ← folder tambahan yang tidak perlu  
    api/  
    public/  
    vercel.json  
    ...
```

Kita perlu struktur ini:

Struktur yang BENAR

```
teledrive/  
  api/                ← langsung di root  
  public/  
  vercel.json  
  package.json  
  ...
```



Solusi:

Buka folder `deploy-vercel`, lalu

select semua file & folder di DALAMNYA

(`api/`, `public/`, `vercel.json`, dll), lalu drag & drop. Jangan drag folder `deploy-vercel` itu sendiri.



LANGKAH 1.4

Commit & Selesai

Setelah semua file ter-upload, scroll ke bawah:

Form Commit

```
Add a commit message: TeleDrive siap deploy
Email:                  (isi email kamu atau biarkan default)
Name:                   (isi nama kamu atau biarkan default)

[ Commit changes ] ← klik tombol hijau ini
```

Klik "**Commit changes**" — selesai!  Semua file sudah ada di GitHub.

2

Cara Alternatif: Buat File Satu-Satu

Kalau tidak bisa upload folder (HP browser tertentu), buat file satu-satu. File teks kecil bisa di-paste langsung, file besar (teledrive.html) perlu di-upload.

LANGKAH 2.1

Buat file `vercel.json`

Di halaman repository, klik:

Add file



Create new file

Nama file: ketik `vercel.json`

Di area konten, paste kode ini:

vercel.json — Copy & Paste

```
{
  "rewrites": [
    { "source": "/", "destination": "/teledrive.html" }
  ],
  "headers": [
    {
      "source": "/api/(.*)",
      "headers": [
        { "key": "Access-Control-Allow-Origin", "value": "*" },
        { "key": "Access-Control-Allow-Methods", "value": "POST, OPTIONS" },
        { "key": "Access-Control-Allow-Headers", "value": "*" }
      ]
    }
  ]
}
```

Scroll ke bawah → klik **"Commit changes"**



LANGKAH 2.2

Buat file `package.json`

Klik **Add file** → **Create new file**, ketik `package.json`, paste:

```
package.json — Copy & Paste

{
  "name": "teledrive",
  "version": "1.0.0",
  "private": true
}
```

Klik **"Commit changes"**



LANGKAH 2.3

Buat file `.gitignore`

Klik **Add file** → **Create new file**, ketik `.gitignore`, paste:

```
.gitignore — Copy & Paste

node_modules/
.vercel/
.env
*.log
```

Klik **"Commit changes"**



LANGKAH 2.4

Buat folder & file `api/tg-api.js`

Klik **Add file** → **Create new file**, ketik nama file lengkap: `api/tg-api.js`

(GitHub otomatis membuat folder `api/` karena ada tanda `/`)



Tips:

Ketik `api/tg-api.js` di kolom nama file — bukan cuma `tg-api.js`. GitHub akan membuat folder "api" otomatis.

Paste kode ini di area konten:

```
export const config = { runtime: 'edge' };

export default async function handler(req) {
  if (req.method === 'OPTIONS') {
    return new Response(null, {
      headers: {
        'Access-Control-Allow-Origin': '*',
        'Access-Control-Allow-Methods': 'POST, OPTIONS',
        'Access-Control-Allow-Headers': '*',
        'Access-Control-Max-Age': '86400',
      },
    });
  }
  if (req.method !== 'POST') {
    return new Response(JSON.stringify({ ok: false, description: 'Hanya status: 405, headers: { 'Content-Type': 'application/json', 'Acce' }));
  }
  try {
    const payload = await req.json();
    const { bot_token, method, proxy_url } = payload;
    const params = payload.params || {};
    if (!bot_token || !method) {
      return new Response(JSON.stringify({ ok: false, description: 'bot status: 400, headers: { 'Content-Type': 'application/json', 'Ac' }));
    }
    let tgUrl;
    const proxy = (proxy_url || '').trim();
    if (proxy) {
      const base = proxy.endsWith('/') ? proxy : proxy + '/';
      tgUrl = base + 'bot' + bot_token + '/' + method;
    } else {
      tgUrl = 'https://api.telegram.org/bot' + bot_token + '/' + method
    }
    const tgResp = await fetch(tgUrl, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(params),
      signal: AbortSignal.timeout(30000),
    });
    let data;
    try { data = await tgResp.json(); } catch { data = { ok: false, des
    return new Response(JSON.stringify(data), {
```

```
        status: tgResp.status,  
        headers: { 'Content-Type': 'application/json', 'Access-Control-Allow-Origin': '*' },  
    });  
} catch (error) {  
    const msg = error instanceof Error ? error.message : 'API proxy error';  
    return new Response(JSON.stringify({ ok: false, description: msg }), {  
        status: 500, headers: { 'Content-Type': 'application/json', 'Access-Control-Allow-Origin': '*' },  
    });  
}  
}
```

Klik **"Commit changes"**



LANGKAH 2.5

Buat file `api/tg-upload.js`

Sama seperti sebelumnya: **Add file** → **Create new file**, ketik `api/tg-upload.js`, paste:

```

export const config = { runtime: 'edge' };

export default async function handler(req) {
  if (req.method === 'OPTIONS') {
    return new Response(null, {
      headers: {
        'Access-Control-Allow-Origin': '*',
        'Access-Control-Allow-Methods': 'POST, OPTIONS',
        'Access-Control-Allow-Headers': '*',
        'Access-Control-Max-Age': '86400',
      },
    });
  }
  if (req.method !== 'POST') {
    return new Response(JSON.stringify({ ok: false, description: 'Hanya status: 405, headers: { 'Content-Type': 'application/json', 'Acce' }));
  }
  try {
    const contentType = req.headers.get('content-type') || '';
    const body = await req.arrayBuffer();
    const bodyBytes = new Uint8Array(body);
    const boundaryMatch = contentType.match(/boundary=(?:"([^\"]+)"|([^\s;]+));/);
    if (!boundaryMatch) {
      return new Response(JSON.stringify({ ok: false, description: 'Con status: 400, headers: { 'Content-Type': 'application/json', 'Ac' }));
    }
    const boundary = boundaryMatch[1] || boundaryMatch[2];
    const fields = parseMultipartFields(bodyBytes, boundary);
    const botToken = fields.get('bot_token');
    const chatId = fields.get('chat_id');
    const proxyUrl = (fields.get('proxy_url') || '').trim();
    if (!botToken || !chatId) {
      return new Response(JSON.stringify({ ok: false, description: 'bot status: 400, headers: { 'Content-Type': 'application/json', 'Ac' }));
    }
    let tgUrl;
    if (proxyUrl) {
      const base = proxyUrl.endsWith('/') ? proxyUrl : proxyUrl + '/';
      tgUrl = base + 'bot' + botToken + '/sendDocument';
    } else {
      tgUrl = 'https://api.telegram.org/bot' + botToken + '/sendDocumen

```

```

    }
    const tgResp = await fetch(tgUrl, {
      method: 'POST',
      headers: { 'Content-Type': contentType },
      body: body,
      duplex: 'half-duplex',
    });
    let data;
    try { data = await tgResp.json(); } catch { data = { ok: false, des
    return new Response(JSON.stringify(data), {
      status: tgResp.status,
      headers: { 'Content-Type': 'application/json', 'Access-Control-Al
    });
  } catch (error) {
    const msg = error instanceof Error ? error.message : 'Upload proxy
    return new Response(JSON.stringify({ ok: false, description: msg }),
      status: 500, headers: { 'Content-Type': 'application/json', 'Acce
    });
  }
}

function parseMultipartFields(body, boundary) {
  const fields = new Map();
  const boundaryBytes = new TextEncoder().encode('--' + boundary);
  const decoder = new TextDecoder('utf-8', { fatal: false });
  let pos = 0;
  while (pos < body.length) {
    const idx = findBytes(body, boundaryBytes, pos);
    if (idx === -1) break;
    const headerEnd = findDoubleCRLF(body, idx + boundaryBytes.length);
    if (headerEnd === -1) break;
    const headerStr = decoder.decode(body.slice(idx + boundaryBytes.len
    const nameMatch = headerStr.match(/name="([^\"]+)"/);
    if (!nameMatch) { pos = headerEnd + 4; continue; }
    const fieldName = nameMatch[1];
    const nextBoundary = findBytes(body, boundaryBytes, headerEnd + 4);
    if (nextBoundary === -1) break;
    const partData = body.slice(headerEnd + 4, nextBoundary - 2);
    if (['bot_token', 'chat_id', 'proxy_url'].includes(fieldName)) {
      fields.set(fieldName, decoder.decode(partData));
    }
    pos = nextBoundary;
  }
  return fields;
}

```

```
function findBytes(h, n, s) {
  for (let i = s; i <= h.length - n.length; i++) {
    let f = true;
    for (let j = 0; j < n.length; j++) { if (h[i+j] !== n[j]) { f = false; } }
    if (f) return i;
  }
  return -1;
}

function findDoubleCRLF(d, s) {
  for (let i = s; i < d.length - 3; i++) {
    if (d[i]===13 && d[i+1]===10 && d[i+2]===13 && d[i+3]===10) return i;
  }
  return -1;
}
```

Klik "**Commit changes**"



LANGKAH 2.6 ★ PENTING

Upload file `public/teledrive.html`

File ini **436 KB** — terlalu besar untuk di-paste. Gunakan **Upload files**:

Add file



Upload files



STRUKTUR FOLDER PENTING!

File `teledrive.html` HARUS berada di dalam folder `public/`.

Cara benar:

Di file manager HP kamu, buat folder bernama `public` dulu, taruh `teledrive.html` di dalamnya. Lalu upload folder `public` ke GitHub.

Kalau HP tidak bisa buat folder:

Upload `teledrive.html` saja dulu, lalu di GitHub edit nama file menjadi `public/teledrive.html`.

Setelah upload:

Struktur akhir di GitHub harus seperti ini:

```
teledrive/                                ← nama repository
├─ vercel.json
├─ package.json
├─ .gitignore
├─ api/
│   └─ tg-api.js
│   └─ tg-upload.js
└─ public/
    └─ teledrive.html ← ★ FILE UTAMA
```

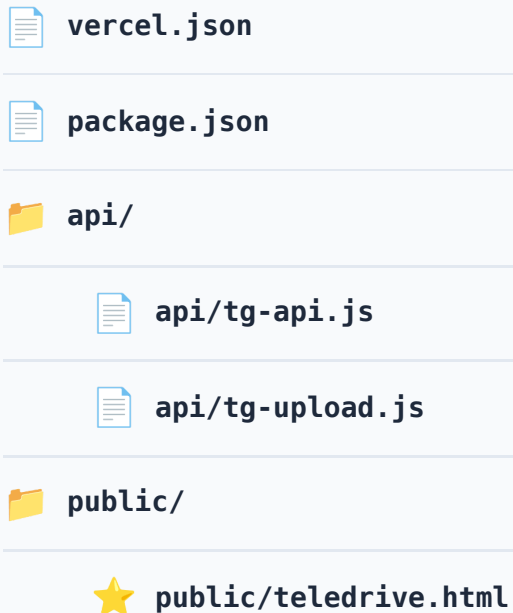
3

Verifikasi & Lanjut ke Vercel

LANGKAH 3.1

Cek Struktur File di GitHub

Buka halaman repository kamu. Pastikan semua file ada dan posisinya benar:



```
vercel.json
package.json
api/
  api/tg-api.js
  api/tg-upload.js
public/
  public/teledrive.html
```

The diagram illustrates the file structure of the repository. It starts with a root level containing `vercel.json` and `package.json`. Below these is the `api/` directory, which contains `api/tg-api.js` and `api/tg-upload.js`. At the bottom is the `public/` directory, which contains the file `public/teledrive.html`. Each file or directory is represented by an icon: a document for files and a folder for directories.



✓ SIAP DEPLOY!

Sekarang Lanjut ke Vercel

File sudah lengkap di GitHub. Sekarang deploy:

1. Buka vercel.com → login dengan GitHub
2. Klik **"Add New → Project"**
3. Pilih repository **teledrive**
4. Framework Preset: pilih **"Other"**
5. Build Command & Output Directory: **kosongkan**
6. Klik **"Deploy"** → selesai!



Tips Khusus untuk HP

Upload File dari HP Android

1. Download `deploy-vercel.zip` dari preview panel
2. Buka app **File Manager** → cari file ZIP
3. Extract ZIP → akan muncul folder `deploy-vercel`
4. Masuk ke folder `deploy-vercel` → select semua item di dalamnya
5. Di browser, buka GitHub → Upload files → pilih file yang tadi di-select

Upload dari iPhone

1. Download `deploy-vercel.zip`
2. Buka app **Files** → tap file ZIP → extract
3. Buka folder hasil extract
4. Di browser (Safari/Chrome) → GitHub → Upload files
5. Tap "**Choose files**" → pilih file satu per satu



iPhone: tidak bisa upload folder sekaligus.

Kamu harus upload file satu per satu atau gunakan Cara 2 (buat file manual di GitHub).

Tips Cepat: Gabung Cara 1 & 2

Cara paling efisien dari HP:

1. File kecil (`vercel.json`, `package.json`, `.gitignore`, `api/*.js`) → **paste langsung** di GitHub web
2. File besar (`teledrive.html`) → **upload file** dari HP

Dengan cara ini kamu tidak perlu extract ZIP sama sekali — cukup copy-paste kode dari tutorial ini!

TeleDrive — Tutorial Upload Manual ke GitHub

Bisa dilakukan 100% dari HP browser • Tidak perlu install apapun